

Lecture 1

1.1 Definitions

1. SQL: Structured Query Language
2. DDL: Data Definition Language (建表)
3. DML: Data Manipulation Language (access, update)

Lecture 2 Intro to SQL and Relation Model

2.1 Relation Schema and Instance

(1) $A_1, A_2 \dots$ attributes, $R = (A_1, \dots, A_n)$ relation schema

$r(R)$ a relation instance r defined over schema R
an element of t of r is called a tuple (row)

Domain (of an attribute) is the allowed values of a A

Atomic: indivisible e.g. full name $\rightarrow$ surname, first name

(2) Null in Attributes: unknown $\neq 0$

Relations are unordered.

Database Schema = relation schemas + constraints

(3) Keys: K is a superkey of R if values for K are unique
if K is a superkey, $\forall K'$ superset of K where $K' \in R$ is also

If K is a superkey, and K is minimum $\Rightarrow$ candidate key

One of candidate keys selected is Primary key.

Foreign key: must appear in another relation

2.2 Intro to SQL

(1) Categories: ① DML create, alter, drop

② DDL select, insert, delete, update

(2) Constraints: ① Not null ② primary key: not null, unique

③ unique: if multiple, unique (A_1, A_2)

④ check (boolean expr) (static, if change, add)

⑤ foreign key ($A_m \dots A_n$) references r

insert into t values ('xx', 'xxx'); delete from movies;
drop table t ; alter table t add A D; alter table t drop column A ;
'o'relly' two ' represent 1 ' in the content

Lecture 3 Basic SQL

3.1 Logical connections: and, or, not

in (), between xxx and xxx (包括两端)

like '%A_': % $\rightarrow$ any, include none; _ $\rightarrow$ 1 char

3.2 Functions

id t ; abc || def $\rightarrow$ concatenate to abcdef; trim ()

upper (); lower (); round (); trunc (); 'acc'

substr ('Citizen Kane', 5, 3) $\dots$ zen; replace ('abc', 'b', 'c')

cast (col as D); case (expr) when ... then ...

is null; is not null when ... then ... end (else)

3.3 distinct: select distinct A_1, A_2 from t

tuple (A_1, A_2) will be distinct.

3.4 Aggregate function

(for primary key)

Calculation on a set of data: Group by, 分组聚合

Aggregate + group by $\Rightarrow$ re group 预分组 take time

count (), min (), max (), sum (), avg () Ignore null

select on aggregate result $\Rightarrow$ select * from (select ...) / having

select count (*) from (

select people_id, count (*) from (

select distinct people_id, credit_as from

credits where credit_as in ('A', 'D')

) group by people_id having count (*) = 2

3.5 Join

(1) Natural join

Same name column join (Bad idea)

(2) Self join

Same table duplicate

(3) Subquery

Subquery using aggregate, then join virtual table

(4) Chaining joins together

join A on ... join B on ... join C on ...

Lecture 4 Intermediate SQL

4.1 More on Join (注意去重 Distinct)

Inner join: default;

Outer join: left: 左连, 右无null; right: 右连, 左无null; full: 全连

4.2 Set operators (自动去重)

(1) UNION

select * from t_1 UNION select ... from t_2

2 select must return same # of columns

data type must be the match

(2) Intersect and Except

① 查询两个结果中都出现的 entries (same column)

② 查询两个结果第1个出现, 第2个不出现的 entries

4.3 Subquery

(1) after select

replace join select m.title, (select c.country_name from country c
where c.country_code = m.country)

(2) after where select * from movies where id in (select id
from country)

in 会自动去重, if contains null X

4.4 Null

most expr with null will be Null

except for True or null = True and false and Null = False

4.5 Ordering

(1) order by $\forall$ col from result set

(2) join: can order by any col from all tables joined

(default)

(3) 多列, asc, desc

(4) Case ... when: order by case type when 'action' then 1 when 'comedy' then 2 else 3

(5) Limit m offset n end,

限制返回结果, 跳过n行

4.6 Window Function

<function> over (partition by <col> order by <col1, col2...>

can be ranking window function or aggregation function

rank() 组内行编号, 重复是此号, dense-rank() 重复是此号 1, 1, 2...

row-number() 每行(编号) by year (as rk from movies) where rk < 3

每行最老三电影 select * from (select *, row_numbers over (p by Country order

4.7 Update and delete

update table.name set column = xxx where ...

delete from table.name where ... 删行

Lecture 5 ER model

5.1 Entity set: student = (ID, name, gpa)
entity attributes

relationship: (12345, 5) ∈ advisor
SID TID

各自引用 Entity sets of a R need not be distinct

5.2 Cardinality (at least 1)

→ 1 ; — many ; = total participation

l...h, l is minimum, h is maximum, * no limit

l=1 ⇒ total participation; h=1 ⇒ at most 1 R

5.3 Primary key

1-to-1: 一对一; 1-to-Many: 一对多; M-to-M: 多对多; 外键与主键

5.4 Weak Entity set

double diamond for identifying relationship

discriminator use dashed line sec-id

+ Strong entity primary key = weak primary key

5.5 Relation schema

强, student (ID, name, gpa); 弱, 带上虚线的主键

composite ⇒ flatten; Multivalued ⇒ multiple schemas

M-to-M: 主键; M-to-1, if M is total, 在M加外键

1-to-1: 任意一边加外键; 1-to-1 identifying R is redundant.

5.6 Relation attributes

Multiple: ① weak entity set ② Composite Attributes }

5.7 1, 2, 3 NF

(1) 1NF: All attributes are atomic

(2) 2NF: Composite PK, every non-key attribute must rely on whole PK

instead just a part of it (single PK is 2NF if 1NF)

(3) 3NF: Non transitive dependence on PK, must directly

Every non key attribute must provide a fact about the key, the key.

Lecture 6. Relational Algebra

6.1 HTML: HyperText Markup Language HTTP: Hypertext Transfer Protocol

REST: Representation State Transfer JSON: JS Object Notation

6.2 Select Operation

$\sigma_p(R)$ p: selection predicate (condition), R: table

Λ and ; V OR ; ¬ not; e.g. $\sigma_{id=1, age \geq 30}(\text{teacher})$

6.3 Project operation

$\pi_{A_1, \dots, A_k}(R)$: 选出特定几列, 去重

$\pi_{name}(\sigma_{id \neq 1}(\text{teachers}))$ Composition

Cartesian product $R \times S$ all possible combinations of rows

6.4 Join operation

$\sigma_{t.id=s.id}(\text{teacher} \times \text{student}) = \text{teacher} \bowtie_{t.id=s.id} \text{student}$

6.5 Union operation, Intersection operation, Difference operation

Union: $R \cup S$ 同列数 (arity), 可比较 (或) (合并去重)

$\pi_{course}(\sigma_{year=2017}(\text{section})) \cup \pi_{course}(\sigma_{year=2018}(\text{section}))$

Intersect: 同 union, $\cap$, $\Join$

Difference: 同 Union, $-$, $r-s \Rightarrow$ 在 r 不在 s

6.6 Assignment, Rename operation

Assignment (临时变量): $\text{Phy} \leftarrow \sigma_{dept='physics'}(\text{teachers})$

Rename: $\rho_{newname}(expr)$ or $\rho_{newname}(A_1, \dots, A_n)(Expr)$ 别名

6.7 equivalent

(when c only involve R)

① $\sigma_A(\sigma_B(R)) = \sigma_B(\sigma_A(R))$ ② $\sigma_C(R \bowtie S) = \sigma_C(R) \bowtie S$

③ $\pi_A(\pi_B(R)) = \pi_A(R)$ if $A \subseteq B$ ④ $(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$

Lecture 7 Normalization

7.1 Decomposition

Lossy: $R = R_1 \cup R_2$, R_1, R_2 are decomposition

$\pi_{R_1}(r) \bowtie \pi_{R_2}(r) = r$ If lossy $\Rightarrow r \subset \pi_{R_1}(r) \bowtie \pi_{R_2}(r)$

Ghost tuple!

7.2 Normalization Theory

$R \xrightarrow{\text{decompose}} \{R_1, \dots, R_n\}$, R not good, $R_1, \dots, R_n$ good

7.3 Functional Dependency

$\alpha \subseteq R, \beta \subseteq R, \alpha \rightarrow \beta$ if $t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$

Closure: F^+ is closure of F, All FD implied by F

F^+ is a superset of F; Trivial: $\alpha \rightarrow \beta$ if $\beta \subseteq \alpha$ then trivial

$K \rightarrow R \Rightarrow K$ is superkey; minimum superkey $K \rightarrow R$ candidate key

∀ candidate key is primary key

If $R_1 \cap R_2 \rightarrow R_1$ or $R_2 \parallel R_1 \cap R_2$ is the superkey for R_1 or R_2

↳ lossless decomposition. necessary when constraints are functional

7.4 Dependency preservation

R 's FD can be check in $R_1, \dots, R_n$ without join.

7.5 BCNF, 3NF

For F^+ of an R,

BCNF: $\alpha \rightarrow \beta$ at least 1 holds α is a superkey of R

if $\alpha \rightarrow \beta$ cause violation in R

$\Rightarrow$ decompose to $(\alpha \vee \beta), (R - (\beta - \alpha))$ NOT dependency-preserving

3NF: $\alpha \rightarrow \beta$ at least 1 holds $\alpha \rightarrow \beta$ is trivial

BCNF $\rightarrow$ 3NF $\Downarrow$ dependency-preservation

Each attribute in $\beta - \alpha$ is contained in a candidate key for R

repetition and null

Lecture 8 Normalization 2

8.1 Armstrong's Axioms

Reflexive: $\beta \subseteq \alpha \Rightarrow \alpha \rightarrow \beta$

Augmentation: $\alpha \rightarrow \beta \Rightarrow \alpha\gamma \rightarrow \beta\gamma$

Transitivity: $\alpha \rightarrow \beta, \beta \rightarrow \gamma \Rightarrow \alpha \rightarrow \gamma$

* Union: $\alpha \rightarrow \beta, \alpha \rightarrow \gamma \Rightarrow \alpha \rightarrow \beta\gamma$

Decomposition: $\alpha \rightarrow \beta\gamma \Rightarrow \alpha \rightarrow \beta, \alpha \rightarrow \gamma$

Pseudotransitivity: $\alpha \rightarrow \beta, \gamma\beta \rightarrow \delta \Rightarrow \alpha\gamma \rightarrow \delta$

8.2 Closure of attribute sets

$R = (A, B, C)$ $F = (A \rightarrow B, B \rightarrow C)$ $A^+ = \{A, B, C\}$

Test for Superkey, $\alpha^+ \rightarrow R$?

Test FD: $\alpha \rightarrow \beta$? check $\beta \subseteq \alpha^+$

Closure F^+

8.3 Canonical Cover

$F_c^+ = F^+$, $F_c \subseteq F$ 删 Extraneous Attributes

Strong constraint 删 LHS 变量, 删 RHS 变量

Left: A is extra in α if $A \in \alpha, F \rightarrow (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha - A \rightarrow \beta\}$

Right: A is extra in β if $A \in \beta, (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow \beta - A\} \rightarrow F$

Test: $\alpha \rightarrow \beta$, in α , $\gamma = \alpha - \{A\}$, check if $\gamma^+ \rightarrow \beta$

A in β , $F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$

check α^+ contains A, if it does, A is extra

8.4 Canonical Cover

$F_c^+ = F^+$ ① no extra ② LHS unique $x \rightarrow y_1, x \rightarrow y_2 \rightarrow x$

$F_c = F$
repeat

Use the union rule to replace any dependencies in F_c of the form $\alpha_1 \rightarrow \beta_1$ and $\alpha_1 \rightarrow \beta_2$ with $\alpha_1 \rightarrow \beta_1\beta_2$.

Find a functional dependency $\alpha \rightarrow \beta$ in F_c with an extraneous attribute either in α or in β .

/* Note: the test for extraneous attributes is done using F_c , not F^+ */

If an extraneous attribute is found, delete it from $\alpha \rightarrow \beta$ in F_c .

until (F_c does not change)

Lecture 9 Storage System and Structure

9.1 Classification of Storage

Volatile: loss data when power off

non-volatile: keep data (battery-back up mem)

Buffering, Read-ahead, Disk-arm-scheduling

9.2 Column-Oriented-Storage

IO = {1, 2, 3, ...} reduce IO, cache performance, compression

vector processing ↑ tuple reconstruction ↓ delete, update cost ↑

9.3 Data dictionary meta data

Name of R; Name, type, length of A; constraints

Lecture 10 Indexing

10.1 Taxonomy

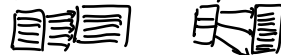
① Integrated: External index & record 存一起?

Mysql PK postgresql cll

② Clustered: Non-clustered index → 物理顺序?

PK Non-PK
Bucket → record must dense

③ Dense: sparse Every search key correspond?



④ Multikay: Single key search key contains 2/ Attributes?

10.2 Implementation

① B-tree order m (max children)

of child = # of keys + 1; ordered, multinary

all leaves same level

$h \leq \lceil \log_{m/2} \lceil \frac{n+1}{2} \rceil \rceil$ n: record #

(i) search similar to AVL tree

(ii) insert 未找到位置, 加, 若上溢出该 node $\lceil \frac{m}{2} \rceil$ 上移, 再处理
if root 溢出, $h+1$ (开始)

(iii) delete 删非叶子 → 删该节点前驱或后继叶子

删叶子 / 无下溢出

下溢出 { 兄弟够借: 借
不够借: 合并 (父节点可能下溢出)
非叶节点合并也要移动子树

② B+ tree order m { key 范围 leaf $\lceil \frac{m-1}{2} \rceil$ to $(m-1)$
基本同 B tree, index only in leaf nodes, linked list on leaf

Insert

1. If the count of key in Leaf Node $< M-1$, we can insert directly.

2. If the count of key in Leaf Node $= M-1$, we should split the Leaf Node to two Leaf Node, and then insert the first key of the Right Leaf Node into the super Node.

- Left Leaf Node contains the index: $1 \sim \lfloor M/2 \rfloor$
- Right Leaf Node contains the index: $\lfloor M/2 \rfloor + 1 \sim M$

3. If the count of key in Leaf Node $= M-1$, we should split the Leaf Node, and then if the count in super Node $= M$, we should also split the super Node to two Interval Nodes.

- Leaf Interval Node contains the index of super Node: $1 \sim \lfloor M/2 \rfloor$
- Right Interval Node contains the index of super Node: $\lfloor M/2 \rfloor + 2 \sim M$
- Insert the $\lfloor M/2 \rfloor + 1$ th key into the super level of the current super Node.

In this way, if the root node is split, then the height of the B+ tree increases by 1.

Delete

1. If the count of key in Leaf Node $> \lfloor M/2 \rfloor - 1$, we can delete directly.

- If the delete value is the first key of current node, we need update the corresponding key in super node to its new first key.

2. If the count of key in Leaf Node $= \lfloor M/2 \rfloor - 1$, and the count of key in its brother node is $> \lfloor M/2 \rfloor - 1$, we should borrow a key from its brother Node. Then change the corresponding key in super node.

3. If the count of key in Leaf Node $= \lfloor M/2 \rfloor - 1$, and its brothers don't have enough keys, we need to merge two Leaf node into one. And then, the count of key in super node should decreased by 1.

4. Continue to scenarios 3, relocated the current node to its super node. If the count of key in current node $< \lfloor M/2 \rfloor - 1$, and the count of key in brother node in its brother node is $> \lfloor M/2 \rfloor - 1$, then:

- Move down the key in super node.
- Move up the key in its brother node.

5. Continue to scenarios 4, if the brother node don't have enough keys, we need to merge two node into one. Continue it to the root node, if the count of the root node is 0 after merge, the height of B+ tree will be decreased by 1.

(1) $\Pi_{title} (\sigma_{movies.country = countries.country_code \wedge countries.country = "China"} (movies \times countries))$

(2) $\Pi_{title} (\sigma_{countries.country = "China"} (movies \bowtie_{movies.country = countries.country_code} countries))$

(3) $\Pi_{title} (movies \bowtie_{movies.country = countries.country_code} (\sigma_{countries.country = "China"} (countries)))$

- In (1), a full Cartesian product will be computed, which costs huge time for matching all pairs and massive temporary storage space for the intermediate product table
- In (2), a smaller intermediate join table is to be cached, which saves some space
- In (3), the filter ($\sigma_{countries.country = "China"}$) reduces the size of the right table in the join operation, which saves a lot of time for pair matching and caching intermediate join table

• Push projections using equivalence rules 8a and 8b; eliminate unneeded attributes from intermediate results to get:

$\Pi_{name, title}(\Pi_{name, course_id}(\sigma_{dept_name = 'Music'}(instructor \bowtie teaches)) \bowtie \Pi_{course_id, title}(course)))$

10.3 Index or not

Yes: PK, unique, where(...), function

小表用 full scan 不差

Lecture 11 Query Processing

11.1 Basic steps

S ms ps ns ps

① Parsing and Translation ② Optimization ③ Evaluation

different evaluation plan, choose lowest cost

$$Cost = b \times t_T + S \times t_s$$

#block transfer / block #seek seek / time

11.2 select operation

① Basic seq scan

$$Cost = \frac{b_T}{\#block} \times t_T + \frac{1}{\#seek} \times t_s$$

② Index scan

① Clustered Index Scan

(i) 等值, search key is candidate $h_i = B^+$ tree height

$$cost = (h_i + 1)(t_T + t_s) \text{ 从根向下找}$$

(ii) 等值, not candidate

$$cost = h_i(t_T + t_s) + t_s + b_T \text{ 找到第1个 } A = v \text{ leaf 顺序向后, 直到 } A \neq v$$

② Secondary Index Scan

(i) 等值, candidate h_i index block, 1 leaf block

$$cost = (h_i + 1)(t_T + t_s) \text{ 1 random I/O to record.}$$

(ii) 等值, not candidate (n item) $\rightarrow$ n leaf blocks

$$cost = (h_i + n)(t_T + t_s) \text{ when n big, slower than full scan}$$

③ Comparison

(i) clustering: 定位到第1个 $\geq v$ leaf, read to end
扫描到第1个 $> v$ leaf, 返回前面 (do not index)

(ii) non clustering: $\forall$ need scan on leaf pointer
(no index needed) scan anyway

④ Conjunction $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$

(i) single key indexes: choose min cost θ_i ,

add res to buffer in mem, then test other

(ii) intersection of identifiers: each $\theta_i \rightarrow$ get indexes
then do intersection, then fetch

⑤ Disjunction $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$

if all θ_i has index; otherwise full scan.

⑥ Bit map: block access bucket $\Rightarrow$ index scan set 1.

Seq scan read 1 $\Rightarrow$ index, 多 1 $\Rightarrow$ Seq

(3) Join

① Nested-Loop join or $(b_r + b_s)t_T + 2t_s$ 对 r 读进 mem

(i) no index $cost = (b_r \cdot b_s + b_r)t_T + (1 + b_r)t_s$

(ii) Index (等值 join) 乘 内表必须有 index (S)

$$cost = b_r(\text{Index scan}) + b_r(t_T + t_s) \text{ 小表驱动大表}$$

② Merge join (等值且排序在 join col 上) 双指针

$$cost = (b_r + b_s)t_T + (\lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil)t_s + sort$$

(内存差下排) buffer size (# of block)

③ Hash join (等值): 小表 scan $\Rightarrow h(\text{keys})$ 桶

扫大表, tuple 在桶中查找 $cost = b_r + b_s \text{ look up}$

Lecture 12 Optimization

12.1 Equivalence Rule

(1) $\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$ (2) $\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(E))$

(3) $\Pi_{L_1}(\Pi_{L_2} \dots (\Pi_{L_n}(E))) \equiv \Pi_{L_1}(E)$ 若 $L_1 \subseteq L_2 \subseteq \dots \subseteq L_n$

(4) $\sigma_{\theta}(E_1 \times E_2) \equiv E_1 \bowtie_{\theta} E_2$, $\sigma_{\theta_1}(\sigma_{\theta_2}(E_1 \times E_2)) \equiv E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$

(5) $E_1 \bowtie E_2 \equiv E_2 \bowtie E_1$, (6) $(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$ 若 θ_1 only

$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 \equiv E_1 \bowtie_{\theta_1 \wedge \theta_2} (E_2 \bowtie_{\theta_3} E_3)$ 有 E_2, E_3

(7) $\theta_0 \rightarrow E_1 \Rightarrow \sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$

$\theta_1 \rightarrow E_1, \theta_2 \rightarrow E_2 \Rightarrow \sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) \equiv \sigma_{\theta_1}(E_1) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$

(8) $\theta \rightarrow L_1 \vee L_2 \Rightarrow \Pi_{L_1 \vee L_2}(E_1 \bowtie_{\theta} E_2) \equiv \Pi_{L_1}(E_1) \bowtie_{\theta} \Pi_{L_2}(E_2)$

$L_3 \not\subseteq L_1 \vee L_2$ from E_1 , $L_4 \not\subseteq L_1 \vee L_2$ from E_2

$\Pi_{L_1 \vee L_2}(E_1 \bowtie_{\theta} E_2) \equiv \Pi_{L_1 \vee L_2}(\Pi_{L_1 \vee L_3}(E_1) \bowtie_{\theta} \Pi_{L_1 \vee L_4}(E_2))$

(9) $E_1 \vee E_2 \equiv E_2 \vee E_1$, $E_1 \wedge E_2 \equiv E_2 \wedge E_1$, (10) $E_1 \vee E_2 \vee E_3$, $E_1 \wedge E_2 \wedge E_3$ ()

(11) $\sigma_{\theta}(E_1 \vee E_2) \equiv \sigma_{\theta}(E_1) \vee \sigma_{\theta}(E_2)$

$\sigma_{\theta}(E_1 \wedge E_2) \equiv \sigma_{\theta}(E_1) \wedge \sigma_{\theta}(E_2) \equiv \sigma_{\theta}(E_1 \wedge E_2)$

$\sigma_{\theta}(E_1 - E_2) \equiv \sigma_{\theta}(E_1) - \sigma_{\theta}(E_2) \equiv \sigma_{\theta}(E_1) - E_2$

(12) $\Pi_{L_1}(E_1 \vee E_2) \equiv (\Pi_{L_1}(E_1)) \vee (\Pi_{L_1}(E_2))$

12.2 Methods

(1) Pushing selections (2) Pushing Projections (3) 生 join 表

Lecture 13 Transaction

13.1 ACID:

Atomicity: 全执行或一个都不执行, rollback!

Consistency: 满足约束, 前后一致

Isolation: 并发执行不能相互干扰, 每个事务认为自己独立

Durability: commit $\rightarrow$ 数据保留

13.2 Transaction State

active $\begin{cases} \nearrow \text{partial commit} \rightarrow \text{committed} \\ \searrow \text{failed} \rightarrow \text{aborted} \end{cases}$

13.3 Schedule

value change at write, flush into disk when commit

serial can be transformed to.

13.4 Serializable $\Rightarrow$ if equivalent to a serial schedule

简单 only read, write

Conflict equivalent: $S \rightarrow S'$, swap only non-conflict ins

$S \downarrow$ to a serial schedule then serializable

read, read 不冲突, 其他均冲突

$T_i \rightarrow T_j$ 若 i 先发生 & 冲突

Then precedence graph is acyclic $\rightarrow$ top sort

13.5 Recoverable Schedule 可恢复调度

T_1 write T_2 read, T_2 commit must after T_1 commit

13.6 Weak consistency: Tradeoff, not serializable OK